RESEARCH AND LANGUAGE DESIGN

Mosaic

Mathematical Intent, Explicit Obligations, and Kernel-Checked Proof

A proof-language design study grounded in ProveIt and Leant

Central proposal

Let the author write the mathematical architecture of an argument. Let a bounded, extensible elaborator reconstruct its routine details. Require both the architecture and the reconstruction to survive independent formal checking.

Prepared for Vladimir Reshetnikov
Research snapshot: September 5, 2026
This is a language and implementation proposal, not a report of a completed compiler. All Mosaic programs are illustrative syntax. Mathematical examples are worked out in the article; no Lean compilation or performance experiment is claimed.

Abstract

A useful mathematical proof language should not merely abbreviate tactic scripts. It should preserve the intermediate assertions, choices, reductions, and invariants that explain why a theorem is true, while delegating representation changes and routine consequences to proof-producing automation. This article proposes Mosaic, a contract-directed front end to Lean. Its three synchronized representations are a human-readable proof spine, a typed graph of obligations, and a collection of kernel-checkable certificates. An omitted justification is not an axiom: it is a precisely scoped reconstruction problem with an explicit resource budget and a visible failure state.

The design is grounded in selected source files from ProveIt and in Leant's separation of candidate generation, checking, suggestion, and evidence bookkeeping. Worked examples cover a factorial-strength Fabius-function bound, a sharp Lipschitz constant, a symmetry reduction, a floor-square-root sum, and structural term synthesis. They expose the difference between administrative verbosity and genuinely necessary hypotheses. We give a small formal semantics and a conditional soundness argument, specify statement locking and proof-replay policies, and explain why kernel acceptance alone cannot guarantee that a formal statement expresses the author's intended mathematics. Related work includes Mizar, Isar, miz3, Naproche, Verbose Lean, Waterproof, and the 2026 Visored proposal. The contribution is a concrete integration of these concerns for research-oriented Lean development, not a claim to have invented declarative proof or natural-language formalization. The article closes with an incremental implementation plan and an evaluation design that charges for hidden automation, adapters, and maintenance rather than counting only short surface proofs.

Design status. The two repositories were inspected at pinned revisions. The investigation is a source-level design study of a small, deliberately selected sample, not a repository-wide proof audit. The supplied archive contains this article and its self-contained LaTeX source, together with build instructions and a source manifest. It does not contain an implemented Mosaic compiler or newly machine-checked versions of the case studies.

Contents

1 The problem is not simply that proofs are long

Consider three ways to present the same proof. The first is a page of low-level rewrites, coercions, and tactic invocations. The second is the single command "search until this closes." The third says: use induction; apply the induction hypothesis at a rescaled argument; integrate the resulting inequality; simplify the coefficient. The second is shortest, but the third best explains the argument. A successful new language should optimize for the third.

The objective is therefore *explanatory compression*: retain decisions that carry mathematical information, and reconstruct details whose form is largely forced once those decisions are fixed. This is a design objective, not a claim that a unique boundary between "important" and "routine" exists. An analyst may consider integrability obvious from continuity; a novice may need that implication displayed. A researcher investigating constructive content may regard a choice principle as central even when another reader does not. The interface must support different levels of disclosure without changing the theorem.

Lean already distinguishes a proof-generating tactic layer from its type-theoretic checking layer; tactics need not be trusted merely because they help construct a proof term [12]. The proposal here is to exploit that separation more systematically at the level of mathematical discourse. A proof author should usually name the intended operation and the assertion it establishes, rather than the exact implementation sequence that makes a particular library lemma applicable.

1.1 What should disappear, and what should remain?

There are at least four different sources of proof length. *Logical structure* includes assumptions, case distinctions, witnesses, induction motives, and the connection between lemmas. *Mathematical infrastructure* includes theorems about integration, cardinalities, or derivatives. *Representation work* reconciles equivalent encodings, such as a nonnegative-real Lipschitz constant and an ordinary real inequality. *Search residue* records experiments that found a workable rewrite order or a library declaration.

The first category should normally remain visible. The second should usually be named through a mathematical interface. The third should often be reconstructible, but never semantically erased. The fourth should be retained as optional provenance rather than becoming the primary published proof.

For example, the distinction between natural and rational division is not typographical noise. Nor is the distinction between a property holding at every point and holding almost everywhere. A language that silently identifies these notions has not made mathematics easier; it has changed the statement. The right abstraction is a *proved bridge* whose applicability conditions are themselves checked.

1.2 A precise design target

The working name Mosaic denotes a proposed language with the following contract:

> An accepted document fixes a formal statement, validates every asserted checkpoint in its declared scope, reconstructs all omitted justifications, and exports a proof of that exact statement under an explicit foundational policy.

"Accepted" is intentionally stronger than "the final goal happened to be solved." If a document contains a false intermediate assertion and an automated prover bypasses it to prove the final result, the document must still be rejected. Conversely, a true but irrelevant assertion may be checked and marked unused; its presence does not make the final proof more explanatory.

The language should be introduced as a Lean extension, not as a replacement foundation. It should interoperate with existing declarations and allow ordinary Lean inside difficult subproofs. The primary engineering risk is not the small surface grammar. It is the quality of the obligation generator, the library interfaces, and the feedback when reconstruction fails.

2 Evidence from the repositories

2.1 Scope and reproducibility

The ProveIt snapshot is commit `24ce8bd743ea`; the Leant snapshot is `c36adf114035`. Their full identifiers and source paths are given in Appendix A. ProveIt's pinned `lean-toolchain` specifies Lean 4.32.0 [1, 2, 3]. Online reference documentation was consulted separately; its evolving "latest" version must not be confused with that repository's toolchain.

The main mathematical sample consists of `SharpFlatness.lean`, `Regularity.lean`, and `FloorSqrtSum.lean`. The Leant sample includes its README, synthesis internals, the fragment translation module, the verification abstraction, and the interactive suggestion code. No whole-project build was run. This is enough to motivate interfaces and identify representative friction, but not to infer average proof lengths, correctness rates, or achievable speedups for either repository.

2.2 A clear mathematical proof surrounded by representation work

In `SharpFlatness.lean`, the declaration

`fabiusReal_le_two_pow_div_factorial_mul_pow`

proves a local upper bound by induction, uses an integral identity, dominates a rescaled integrand, and evaluates a polynomial integral. Around that spine are explicit order, factorial, binomial-coefficient, and cast manipulations. The companion declaration

`rvachevUp_le_two_pow_div_factorial_mul_pow`

transfers the result through an already available identity [4].

The actual instruction `induction n generalizing x with` is already close to mathematical intent. Its generalization clause matters; the repeated conversions around the argument are a different kind of detail.

This example suggests a language of *mathematical transformations*: apply an induction hypothesis at a specified argument, integrate a specified inequality, and normalize a coefficient in a specified domain. It does not suggest that integrability or the domain of the induction hypothesis should be ignored. Section 6 reconstructs the mathematical argument independently and turns those requirements into an obligation ledger.

2.3 An important counterexample to a sweeping diagnosis

`Regularity.lean` contains derivative-to-Lipschitz arguments, a lower bound on any Lipschitz constant obtained at the midpoint, and a sign split for an absolute-value bound. Yet its absolute-difference corollary is already a short application of a Lipschitz estimate. This is evidence for a mixed diagnosis: some friction is representational, some disappears once the right lemma exists, and some proofs are already concise [5].

A fair comparison must therefore include an *idiomatically refactored Lean baseline*. It would be misleading to compare a polished proposed language, equipped with newly written domain

lemmas, only against an unrefactored proof that lacks those lemmas. Sometimes the best immediate improvement is a new theorem in the existing library, not a new language feature.

2.4 The natural-number boundary is genuinely mathematical

`FloorSqrtSum.lean` defines a natural-valued closed form, proves a rational version using a successor-step split, and transfers back to natural numbers. The source separately establishes divisibility by six and a bound ensuring that natural subtraction does not truncate. Its cast lemma uses these facts before the final natural equality is recovered [6].

For example, the cast bridge contains these two instructive lines:

```
rw [Nat.cast_sub (floorSqrtSumClosedForm_subtrahend_le n)]
rw [Nat.cast_div_charZero (six_dvd_sqrt_sum_numer (Nat.sqrt n))]
```

They are not arbitrary ceremony: each cites evidence that a familiar-looking arithmetic operation survives the change of number system.

This is a particularly useful stress test. A proposed command "work over the rationals" must not simply reinterpret all natural operations as field operations. It must generate precisely the conditions under which the reinterpretation is valid. A short presentation that leaves those conditions hidden but discharged is a success; one that makes them disappear from the verification problem is unsound.

2.5 What Leant contributes

Leant's documented workflow separates synthesis from backend checking and supports interactive, checked tactic suggestions. It also exposes a distinction between complete reasoning in a supported fragment and bounded or unsupported search. Its fragment translation preserves some structure while treating term-indexed or other unsupported expressions opaquely [7, 8]. These are useful ingredients for a proof language, but they do not constitute a general solver for analytic or dependent mathematical reasoning.

A particularly instructive implementation detail is the opaque `Verified` wrapper in `Verification.hs`: it records acceptance by a supplied callback, not a kernel proof or a behavioral certificate. The abstraction prevents certain accidental bookkeeping errors, but its name does not create a new trust guarantee [9]. Mosaic should similarly use strong internal types while requiring actual proof artifacts at its acceptance boundary.

The suggestion code distinguishes proposing from committing a tactic and records an explicit replacement when a tactic reports one. The synthesis internals also document careful tracking of candidates and associated evidence [10, 11]. The design lesson is broader than copying a REPL: exploratory search should produce stable, inspectable material, and a user should be able to distinguish a checked result from an attractive proposal.

3 Position among existing proof languages

The basic ambition is established, not new. Mizar offers a formal mathematical vernacular; Isar provides structured, readable proof documents rather than only tactic scripts [14, 15]. Wiedijk's miz3 work explicitly synthesizes declarative presentation with procedural automation, including help in discovering intermediate statements [16]. These precedents rule out the claim that adding words such as "assume," "therefore," or "by" is itself a research contribution.

Naproche accepts controlled mathematical language, including a LaTeX-oriented form, and uses automated reasoning to fill gaps [17, 18]. Massot's Verbose Lean work is especially close to the present setting: it provides natural-language tactics for mathematical teaching in Lean [19, 20]. Waterproof likewise develops a controlled-language environment for learning mathematical proof with a proof-assistant backend [21]. A research-oriented design should learn from these systems rather than treat human-readable formal proof as an unexplored idea.

The June 2026 Visored preprint is an even more recent close neighbor. It describes a mathematical controlled language, typed intermediate representations, and rule-driven reconstruction; Lean emission is presented as an optional downstream stage rather than the system's primary correctness route [22]. Mosaic makes a different architectural commitment: acceptance in its proposed Lean profile always includes an independently checked Lean artifact. This is a distinction between stated design contracts, not an experimental comparison or an audit of Visored's implementation.

At the automation layer, Aesop supplies rule-based proof search, Lean's `grind` combines automated reasoning procedures, and LLMSTEP connects language-model suggestions to checking in Lean [23, 24, 25]. These are potential components. Reimplementing them inside a new language would waste effort and risk losing compatibility with the library ecosystem.

Tradition	Lesson to retain	Additional emphasis here
Mizar and Isar	Assertions and scope can organize a formal proof.	Per-checkpoint obligations and explicit reconstruction policies.
miz3	Declarative text and procedural discovery can cooperate.	Frozen targets, stable replay, and separation of checked text from search history.
Naproche	Mathematical prose can have a controlled formal interpretation.	Lean-native dependent expressions and mandatory Lean acceptance in the chosen profile.
Verbose Lean and Waterproof	Natural mathematical phrasing can guide users.	Research-scale adapters, effect-aware transformations, and auditability.
Leant and search tools	Candidate generation can be separated from acceptance.	Typed mathematical contracts, evidence-preserving composition, and certificate export.

Table 1: Design influences. The final column states this article's emphasis; it is not a claim that the cited systems lack every listed feature.

The contribution proposed here is an integration: mathematical checkpoints with explicit dependency scopes; domain-aware obligation generation; several interchangeable synthesis providers; a frozen-statement and certificate discipline; and an evaluation protocol that includes hidden infrastructure costs. Whether this integration is practically better remains an empirical question.

4 Three synchronized representations of a proof

4.1 The proof spine

The *spine* contains the decisions a reader needs to reconstruct the argument conceptually. It may state a useful lemma, choose an induction variable, exhibit a witness, reduce by symmetry, or announce an integral comparison. A spine is not necessarily prose: a short calculation chain may explain an argument better than a paragraph.

The author need not narrate the chronological discovery process. Failed approaches belong in

a notebook or an optional history, not in the final proof. What should survive is the dependency structure that makes the successful reasoning intelligible. For example, “choose the scale so that the induction hypothesis applies” communicates more than the names of five arithmetic tactics.

4.2 The obligation graph

Every meaningful sentence elaborates to one or more typed goals, with the context in which each goal must be proved. A node records a statement, local variables, permitted facts, the mathematical method requested, a resource policy, and links to the source text. Edges describe actual prerequisite relationships. The graph may branch for cases and join when their conclusions are assembled.

This graph makes implicit mathematics explicit without requiring it all to occupy the printed page. Clicking “integrate” should reveal continuity or integrability requirements, the pointwise inequality, the interval orientation, and the exact theorem instantiated. Clicking “by algebra” should show whether the operation used ring normalization, cancellation under a nonzero condition, or an order argument. These are different contracts.

Not every node is a proof proposition. Elaboration may need to choose a type, a coercion, a type-class instance, or a witness. Such choices must be recorded as *semantic commitments*. A theorem that checks only after silently changing the type of a variable has failed the author’s contract even if the resulting term is well-typed.

4.3 The certificate representation

The final representation contains actual Lean terms or a replay artifact that reconstructs them without invoking open-ended discovery. Every exported theorem must be checked against the frozen target and audited according to the selected axiom policy. The kernel does not need to understand phrases such as “by symmetry”; it needs the ordinary proof obtained by elaborating that phrase.

The three representations should be navigable in both directions. A side condition in a kernel-level explanation should point to the mathematical operation that created it. An author’s checkpoint should point to its term and actual dependencies. A changed imported declaration should identify the affected operations, not merely produce an unexplained failure in generated code.

4.4 A language is more than its keywords

A pure syntax wrapper around tactics would implement only part of this design. The crucial object is a *reasoning contract*. For example, a ring-normalization contract promises equality in a fixed algebraic structure; it does not promise to discover an induction invariant. An interval-comparison contract promises to reduce an integral inequality to an oriented pointwise comparison and integrability hypotheses; it does not promise to find the dominating function.

This division creates a productive boundary. The human contributes the domination, invariant, witness, or key intermediate estimate. The system determines whether the chosen move is legitimate, reconstructs standardized consequences, and reports the smallest unresolved mathematical requirements it can identify.

5 A concrete surface language

All listings in this article are proposed syntax, not working Lean extensions. ASCII notation is used in listings for portable typesetting; an editor could display the corresponding mathematical symbols. Terms inside expressions retain a Lean-compatible typed interpretation. Controlled phrases organize those terms rather than asking a language model to decide their meaning during verification.

5.1 The core constructs

```
theorem example (x : Real) : premise(x) -> target(x)
proof
  assume h : premise(x)
  let y : Real := expression(x)
  have bound : intermediate(x, y)
    by method using h
  suffice reduced : simpler_target(x, y)
    by bridge using bound
  show simpler_target(x, y)
    by method using bound
end
```

The words `have` and `show` create checked assertions. `let` introduces a definition, not an existence claim. `suffice` creates both a new subgoal and an obligation that this subgoal implies the previous target. The latter implication cannot be assumed merely because the new goal looks easier.

`Assume` may introduce a premise only when opening a corresponding implication or an explicitly scoped subproof. It cannot append a convenient global hypothesis to a theorem already being proved. The elaborated statement is fixed before its proof body is searched.

Useful structured forms include `fix`, `obtain`, `cases`, `induction`, `calc`, and `conclude`. Their semantics should be small and explicit. Domain commands such as `integrate`, `transport`, and `count` expand into the core by invoking registered, proved interfaces. They should not become opaque universal tactics with human-sounding names.

5.2 Facts, references, and controlled omission

The following three forms deliberately mean different things:

```
have h : Q by routine
have h : Q by routine using p, r
have h : Q by routine from only p, r
```

The first searches within the ambient declared policy. The second prioritizes named facts but still allows the ambient context. The third restricts local proof inputs to the named facts and the variables and assumptions needed to type them. It does not implicitly ban every imported theorem; a separate `library only` clause can restrict global theorem search.

This distinction prevents a common ambiguity in mathematical exposition. "Using p" may mean "p is helpful," "p is sufficient," or "this argument must genuinely depend on p." These are not equivalent. Mosaic can certify sufficiency by proving Q in the restricted context. It can

report syntactic dependencies in a certificate. It cannot generally certify that a premise was psychologically essential or mathematically indispensable.

Pronouns such as `this` may be permitted only when they resolve to a unique preceding checked statement in the current scope. An ambiguous reference should produce a choice of explicit labels, not be resolved according to whichever interpretation makes a solver succeed. Publication mode should make potentially confusing references explicit.

5.3 Definitions, properties, and quantifier order

A declaration "let f be continuous" combines an object and a property. Its core representation must still distinguish the function f from a proof of its continuity. Likewise, "choose N sufficiently large" is meaningful only after a property of N has been specified and existence proved.

For example, these are different tasks:

$$\forall \varepsilon > 0\ \exists N\ \forall n \geq N\ P(\varepsilon, n), \qquad \exists N\ \forall \varepsilon > 0\ \forall n \geq N\ P(\varepsilon, n).$$

A natural-language parser may offer either reading during drafting, but verification must operate on one selected, visible telescope of binders. Solving the easier reading is not an acceptable method of disambiguation.

An existential witness introduced with `obtain` has lexical scope and the precise property justified by its source theorem. A `let` binding cannot stand in for an unproved existential. In dependent constructions, later types may mention earlier choices; these dependencies must be preserved in the generated Lean context, not flattened into a bag of sentences.

5.4 Calculations, methods, and escape hatches

A calculation chain states the successive expressions and relations. It should preserve the distinction between equality, implication, equivalence, strict inequality, and non-strict inequality. Mixed chains require a certified transitivity rule.

```
calc
  expression_0 = expression_1 by expand definitions
               <= expression_2 by bound using h
               = expression_3 by algebra in Real
```

A `by Lean` block may contain an ordinary Lean term or tactic proof. It inherits the same target, local scope, and axiom restrictions as any other justification. This provides a practical escape hatch without creating a second, weaker notion of acceptance.

Free prose belongs in annotations unless the author explicitly promotes it to controlled syntax. An AI assistant can suggest a promotion and show its formal interpretation. The user's approved structured source, rather than the original ambiguous sentence, is what the verification pipeline accepts.

6 Worked example: the factorial-strength flatness bound

This section develops an original presentation of the argument underlying the first case study. The purpose is to specify what a concise proof would mean, not to claim that the following program has already been compiled. We deliberately separate a reusable analytic lemma from the adapter that supplies its assumptions for the repository's Fabius function.

6.1 The mathematical interface

Let $f : \mathbb{R} \to \mathbb{R}$ be continuous, suppose $f(x) \le 1$ for $0 \le x \le 1$, and assume

$$f(x) = \int_0^x 2f(2t)\,dt \qquad (0 \le x \le \tfrac{1}{2}). \tag{1}$$

Define, with the division and the base of the power interpreted in $\mathbb{R}$,

$$A_n = \frac{2^{\binom{n+1}{2}}}{n!} \qquad (n \in \mathbb{N}). \tag{2}$$

The useful coefficient recurrence is

$$A_0 = 1, \qquad A_{n+1} = \frac{2^{n+1}A_n}{n+1}. \tag{3}$$

It follows from $(n+1)! = (n+1)n!$ and $\binom{n+2}{2} = \binom{n+1}{2} + n + 1$. In particular, the denominator being cancelled is nonzero. The desired result is

$$0 \le x, \quad 2^n x \le 1 \quad \Longrightarrow \quad f(x) \le A_n x^n. \tag{4}$$

The repository-specific part of a future implementation would prove that the chosen $f = \mathrm{fabiusReal}(F)$ satisfies this interface under the existing $\mathrm{IsFabius}(F)$ assumption. It must not introduce these properties as new axioms.

6.2 A complete mathematical derivation

We prove (4) by induction on n, keeping x arbitrary. For $n = 0$, the assumptions give $0 \le x \le 1$, and the result is the assumed bound $f(x) \le 1$ because $A_0 x^0 = 1$.

Suppose the assertion is true at n, and let $x \ge 0$ satisfy $2^{n+1}x \le 1$. Since $2^{n+1} \ge 2$, we have $x \le 1/2$. For $0 \le t \le x$,

$$0 \le 2t, \qquad 2^n(2t) = 2^{n+1}t \le 2^{n+1}x \le 1.$$

Thus the induction hypothesis applies at $2t$ and gives

$$2f(2t) \le 2A_n(2t)^n = 2^{n+1}A_n t^n. \tag{5}$$

Both sides are continuous as functions of t, hence integrable on the compact interval in question. Because $0 \le x$, integration preserves the displayed inequality. Using (1),

$$\begin{aligned} f(x) &= \int_0^x 2f(2t)\,dt \\ &\le 2^{n+1}A_n \int_0^x t^n\,dt \\ &= \frac{2^{n+1}A_n}{n+1}x^{n+1} = A_{n+1}x^{n+1}. \end{aligned}$$

This also covers $x = 0$: the interval integral vanishes, and no division by x is used. The only division in the coefficient calculation is by positive integers interpreted in $\mathbb{R}$.

The important mathematical choice is not a particular rewrite order. It is applying the induction hypothesis at $2t$ and integrating rather than replacing the integrand by a single endpoint bound. The latter loses the coefficient arising from the power integral.

6.3 The proposed source

The interface facts and the coefficient recurrence would be named declarations. A proof spine could then read:

```
theorem flat_bound (n : Nat) (x : Real)
  given hx : 0 <= x, hscale : 2^n * x <= 1
  shows f(x) <= A(n) * x^n
proof
  induction n generalizing x
  case zero
    conclude by bound using unit_bound, A_zero
  case succ n, ih
    have small : x <= 1/2 by order using hscale
    have dom : forall t in [0,x],
      2*f(2*t) <= 2^(n+1)*A(n)*t^n
    proof
      fix t in [0,x]
      have admissible : 0 <= 2*t and 2^n*(2*t) <= 1
        by order using hx, hscale
      conclude using ih at (2*t), admissible by algebra
    end
    integrate dom over [0,x] as integrated
      using continuity(f), hx
    conclude using integrated, integral_law at x,
      small, power_integral, A_successor by algebra
end
```

This listing is not a claim that every line is cheap to implement. In particular, `integrate` needs a library interface, and `conclude` must instantiate the named facts and produce a chain of checked implications. A prototype should first lower those operations to explicit intermediate statements. Only after their obligations are reliable should the presentation be made this compact.

6.4 What the compact source owes the checker

Node	Formal obligation	Intended source of evidence
F1	$\forall x,\ 0 \le x \to 2^n x \le 1 \to f(x) \le A_n x^n$ is the induction hypothesis.	The generalized induction motive, not a hypothesis fixed at the original x.
F2	$x \le 1/2$ in the successor case.	$x \ge 0$, $2^{n+1}x \le 1$, $2^{n+1} \ge 2$, and order reasoning.
F3	$0 \le 2t$ and $2^n(2t) \le 1$ for $t \in [0, x]$.	Interval membership, positivity of powers of two, and the successor scale bound.
F4	The implication from the induction bound to (5).	Multiplication by the nonnegative scalar 2, then algebraic identities.
F5	Both integrands are interval-integrable.	Continuity of f, composition, and polynomial continuity; no blanket integrability assumption.
F6	Integral comparison has the correct orientation.	$0 \le x$ and the actual pointwise or almost-everywhere comparison required by the chosen theorem.

Node	Formal obligation	Intended source of evidence
F7	$f(x)$ equals the left integral.	The integral law with both domain conditions discharged.
F8	The right integral has the stated value.	The power-integral theorem, constant extraction, zero endpoint, and $n+1 \neq 0$ in $\mathbb{R}$.
F9	The resulting coefficient is A_{n+1}.	Binomial and factorial recurrences, cast compatibility, exponent identities, and nonzero denominators.

Table 2: An obligation ledger for the flatness argument. No entry is justified merely by being hidden in the interface.

Some facts can be shared across nodes, but sharing is an optimization. Their validity is not inferred from the fact that several consumers need them. Nor should the solver search for a different final proof and ignore an unsuccessful F3 or F5. Those nodes represent assertions made by the authored document.

6.5 Why induction generalization belongs in the visible layer

If induction is performed after fixing x without generalizing it, the hypothesis may only describe $f(x)$. It then cannot be applied to $f(2t)$. This is not a minor parser issue. It is a change in the induction motive.

A useful assistant can propose generalizing x, and perhaps other parameters that occur in the intended instantiation. But the resulting motive should be shown and stored. It should not be silently strengthened merely because a tactic search otherwise fails. The strengthened statement may itself require additional base or step obligations.

For research proofs, a small amount of visible specificity is valuable. The words `generalizing x` are not verbosity to eliminate. They express the uniformity on which the argument depends.

7 Lipschitz reasoning and certified symmetry

7.1 A reusable mathematical pattern

Suppose a differentiable function $f : \mathbb{R} \to \mathbb{R}$ satisfies $|f'(x)| \leq L$ for all x, with $L \geq 0$. The mean value theorem gives

$$|f(x) - f(y)| \leq L|x - y|.$$

Indeed, when $x \neq y$, apply that theorem on the interval between x and y and take absolute values; the case $x = y$ is immediate. Conversely, if f is K-Lipschitz and differentiable at c, then for $h \neq 0$,

$$\left|\frac{f(c+h) - f(c)}{h}\right| \leq K.$$

Passing to the derivative limit yields $|f'(c)| \leq K$. Therefore, if $|f'(c)| = L$, the Lipschitz constant L is optimal.

For a function satisfying $f'(x) = 2u(2x - 1)$, $0 \leq u \leq 1$, and $u(0) = 1$, these observations give the optimal constant 2, using $c = 1/2$. This is a mathematical derivation of the pattern, not a claim that the derivative theorem or the auxiliary function can be synthesized from their types alone.

```
have upper : Lipschitz(f, 2)
  by derivative_bound using derivative_formula, range_u
have sharp : derivative(f, 1/2) = 2
  by evaluate using derivative_formula, value_u_at_zero
show forall K >= 0, Lipschitz(f, K) implies 2 <= K
  by derivative_of_lipschitz at (1/2) using sharp
```

The relevant interface must distinguish differentiability at a point from a globally defined derivative expression. It must also distinguish a real parameter with a proof of nonnegativity from Lean's nonnegative-real parameterization of a Lipschitz predicate. A view adapter can construct the latter and prove that the displayed inequality is its intended interpretation. It cannot pretend that the two types are definitionally identical.

A library theorem encapsulating the complete optimality pattern may make the eventual Lean proof just as short as the Mosaic proof. That would be a positive result: the language has helped identify a reusable mathematical interface rather than merely conceal a long tactic script.

7.2 What "by symmetry" must establish

Let a domain $D \subseteq X$, a preferred subdomain $D_+ \subseteq D$, and a property $P : X \to \mathrm{Prop}$ be given. One sufficient symmetry-reduction contract consists of a map $r : D \to D_+$ and proofs

$$\forall y \in D_+,\ P(y), \qquad \forall x \in D,\ P(r(x)) \to P(x).$$

No group theory is required for this simple form. A group-action interface is useful when many transformations are involved, but should specialize to the same coverage-and-transport principle.

For a sign reduction on $\mathbb{R}$, suppose $P(-x) \leftrightarrow P(x)$. The domain $[-1, 1]$ is stable under negation, and each point can be moved to $[0, 1]$ by choosing x or $-x$. A command

```
reduce by symmetry x -> -x to 0 <= x
```

therefore owes the checker domain preservation, coverage of the preferred case, and transport of the *entire* property. Symmetry of one expression in the goal is not sufficient if another expression or an assumption breaks the symmetry.

For example, suppose u is even and $u(y) = f(1 - y)$ for $0 \le y \le 1$, with $f(t) \le 2t$ for $t \ge 0$. Then for $|x| \le 1$,

$$u(x) = u(|x|) = f(1 - |x|) \le 2(1 - |x|).$$

The obligations are $|x| \in [0, 1]$, $1 - |x| \ge 0$, and $u(x) = u(|x|)$. The last equality follows by a sign split and evenness. This presentation may be simpler than exposing a separate positive and negative proof, but it has not removed the sign argument from the certificate.

7.3 A deliberately rejected shortcut

From a proof of $x \ge 0 \Rightarrow x = |x|$, one cannot conclude $x = |x|$ for every real x "by symmetry." The property itself is not invariant under negation. Taking $x = -1$ displays the error. A good error report should say that property transport is missing or false, not merely that a general-purpose tactic timed out.

The same issue appears in arguments involving complex conjugation, coordinate permutations, rescaling, and normalization to a unit vector. The transformation must preserve the hypotheses

and supply a way back to the original target. Normalizing by a norm additionally requires treating the zero case. These are reusable proof patterns, not licenses to omit exceptional cases.

8 A discrete example: sums of integer square roots

8.1 The statement and its interpretation

Write $s = \lfloor \sqrt{n} \rfloor$, with $n \in \mathbb{N}$. The intended identity is

$$\sum_{k=1}^{n} \lfloor \sqrt{k} \rfloor = s(n+1) - \frac{s(s+1)(2s+1)}{6}. \tag{6}$$

As an equality in $\mathbb{Q}$, this notation has its usual field interpretation. As an equality in $\mathbb{N}$, division is integer division and subtraction is truncated. The mathematical formula is compatible with both interpretations, but this compatibility must be proved.

The source's induction-by-jumps is a legitimate mathematical approach. A concise version can say that $\lfloor \sqrt{n+1} \rfloor$ either stays at s or rises to $s+1$, and that a rise occurs at $(s+1)^2$. Nevertheless, each alternative and its square-boundary condition must be available as a theorem, not guessed by pattern matching a few numerical examples.

For the language-design exercise, it is also useful to give a *different* proof by double counting. This is a proposed alternative argument, not a description of how the repository's current proof works.

8.2 Double counting with all conditions exposed

Consider the finite set

$$T = \{(j,k) \in \mathbb{N}^2 : 1 \le k \le n,\ 1 \le j,\ j^2 \le k\}.$$

For fixed k, the allowed j are exactly $1, \dots, \lfloor \sqrt{k} \rfloor$. Hence

$$|T| = \sum_{k=1}^{n} \lfloor \sqrt{k} \rfloor.$$

If $(j,k) \in T$, then $j^2 \le n$, so $1 \le j \le s$. Conversely, for each $1 \le j \le s$, the allowed k are $j^2, j^2+1, \dots, n$. This interval is nonempty because $j^2 \le s^2 \le n$, and its cardinality is $n - j^2 + 1$. Thus

$$|T| = \sum_{j=1}^{s} (n - j^2 + 1). \tag{7}$$

To perform the subsequent algebra without ambiguity, regard the summands as integers or rationals. Since $j^2 \le n$ in every summand, this reinterpretation agrees with the corresponding natural expression. We obtain

$$|T| = s(n+1) - \sum_{j=1}^{s} j^2.$$

Finally,

$$\sum_{j=1}^{s} j^2 = \frac{s(s+1)(2s+1)}{6}. \tag{8}$$

For completeness, set $q(s) = s(s+1)(2s+1)$. Direct expansion gives $q(s+1) - q(s) = 6(s+1)^2$, while $q(0) = 0$. Induction therefore proves $q(s) = 6\sum_{j=1}^{s} j^2$, which simultaneously establishes

(8) and divisibility of $q(s)$ by six. The case $n = 0$ is included: $s = 0$, both counting ranges are empty, and both sides of (6) vanish.

The natural subtraction does not truncate because

$$\sum_{j=1}^{s} j^2 \le \sum_{j=1}^{s} (n+1) = s(n+1).$$

Here $j^2 \le n < n+1$ holds for every j in the range, and the empty-range case is harmless. This gives exactly the order fact required for transport back to $\mathbb{N}$.

8.3 How a counting command could work

```
let s : Nat := floor_sqrt(n)
let T := {(j,k) : Nat * Nat |
          1 <= k and k <= n and 1 <= j and j^2 <= k}
have by_columns : card(T) = sum k in [1,n], floor_sqrt(k)
  by count fibers k using floor_sqrt_spec
have by_rows : card(T) = sum j in [1,s], (n - j^2 + 1)
  by count fibers j using floor_sqrt_spec
transport goal to Rational preserving Nat arithmetic
show transported_goal
  using by_columns, by_rows, sum_squares by algebra
```

The displayed set-builder notation describes a finite mathematical set. An implementation must choose a concrete finite representation or supply finiteness evidence; `card` cannot be applied without that representation being resolved. The command `count fibers` should request a dependent-sum decomposition or an explicit finite bijection. It may infer a simple projection, but it must prove the forward and inverse membership conditions and the cardinalities of fibers.

Similarly, `transport` should open a visible side panel containing divisibility, subtraction bounds, and injectivity of the embedding. In this example, those conditions can be supplied by (8) and the preceding bound. The concise command is justified by a proved mathematical account, not by a blanket conversion of all natural arithmetic into rational arithmetic.

8.4 Certified views rather than invisible casts

A *view* should be a typed translation with preservation theorems. For the standard embedding $\iota : \mathbb{N} \to \mathbb{Q}$, addition and multiplication preserve their intended operations without extra hypotheses. By contrast,

$$\iota(a - b) = \iota(a) - \iota(b) \quad \text{requires } b \le a,$$

and

$$\iota(a/b) = \iota(a)/\iota(b) \quad \text{is justified, for } b > 0, \text{ by } b \mid a.$$

These are sufficient contracts; a library may provide more specialized variants. The view mechanism must select a theorem, instantiate it, and discharge its hypotheses.

Unconditional subtraction transport fails already at $a = 0, b = 1$. Unconditional division transport fails at $a = 1, b = 2$. The resulting errors are mathematical, not inconveniences to be hidden behind a more aggressive normalizer.

This example also illustrates the value of an explicit *algebraic domain*. "By algebra in $\mathbb{Q}$" identifies which operations are being normalized. It is stronger and safer than asking an engine to find any interpretation in which the displayed symbolic manipulation goes through.

9 Contract-directed elaboration and synthesis

9.1 An omission is a task, not a truth marker

An omitted justification should be represented as

$$\mathcal{O} = (\mathcal{E}, \Gamma, Q, \mathcal{P}, \mathcal{B}, \mathcal{I}), \tag{9}$$

where $\mathcal{E}$ is the pinned global environment; Γ is the ordered local context; Q is the already elaborated target; $\mathcal{P}$ specifies permitted methods, facts, and axioms; $\mathcal{B}$ bounds resources; and $\mathcal{I}$ records the author's intended mathematical operation. The output must be a proof of Q in this context, a justified decomposition into smaller obligations, or an explicit failure status.

The word "routine" has no logical force. It selects a configurable reconstruction policy. A profile might permit bounded logical synthesis, simplification with a fixed theorem set, linear arithmetic, and ring normalization. Another might additionally permit theorem retrieval or a language-model proposal. Neither profile can accept a proposition merely because the allotted search was confident or almost complete.

9.2 A staged search architecture

The front end should first resolve names, types, binder scopes, and the selected interpretation of notation. It can then attempt definitional equality, direct use of local facts, and structural proof construction. Next come normalization and specialized decision procedures. Theorem retrieval and larger proof searches should use the goal's mathematical shape and the stated intent. AI proposals are another provider, not a privileged final authority.

For an operation `by algebra in Real`, a sensible route is polynomial normalization followed, when needed, by certified cancellation or order reasoning with explicit side conditions. Searching the entire mathematical library for an unrelated theorem would undermine the explanatory purpose of the operation even if it eventually closed the goal. For `by compactness`, the default search space should instead expose a selected compactness theorem and its actual hypotheses.

The stages need not form a rigid global sequence. They can be a budgeted portfolio, provided their outputs obey the same obligation interface. A domain method may first retrieve a lemma, then use logical synthesis to assemble its hypotheses, then invoke arithmetic on a remaining inequality. The important invariant is that each combination produces a checked composition, not an informal assurance that several tools agreed.

9.3 Structural synthesis: powerful at the correct level

Consider the purely logical target

$$(P \to Q \to R) \to (P \wedge Q \to R).$$

A suitable term is

$$\lambda f.\, \lambda h.\, f\,(\mathrm{left}\, h)\,(\mathrm{right}\, h).$$

Once the relevant mathematical facts have been stated as premises, synthesizing this sort of plumbing can spare the author many small introductions, projections, and applications. This is the appropriate role for a Leant-style structural provider. An expression that the provider carries as an opaque atom can still be transported through a logical argument; it is not thereby understood arithmetically.

Now consider the target type $\mathbb{N} \to \mathbb{N}$. It admits many terms that have nothing to do with an intended successor function. Type inhabitation alone does not encode that intention. A more suitable construction contract is

$$\{g : \mathbb{N} \to \mathbb{N} \mid \forall n,\ g(n) = n + 1\}.$$

The equality specification makes the intended behavior part of the proof obligation. It is still possible for search to fail, but success now certifies something meaningful. Type-directed synthesis is most useful when the type or accompanying refinement actually expresses the mathematical requirement.

The same distinction matters for witnesses. Inside a proposition, an existential hypothesis can be eliminated into a local witness and its property. Extracting a globally usable data-valued function from $\forall x\, \exists y\, P(x, y)$ is a different operation; a constructive dependent-pair input and a classical choice construction have different interfaces. A language must record which one it uses rather than hide the distinction behind "choose."

9.4 Do not turn fragment exhaustion into mathematical refutation

A provider's result type should distinguish at least the following outcomes:

```
Solved(checked_artifact)
Refined(child_obligations, checked_combiner)
Exhausted(budget, explored_fragment)
Unsupported(reason)
Refuted(checked_proof_of_negation)
FragmentUninhabited(fragment_certificate, applicability_conditions)
```

The last two outcomes are deliberately different. An exact decision procedure may certify non-inhabitation in its own fragment. That certificate establishes non-provability of the original target only with a justified completeness relationship between the fragment and the original problem. It is generally not a proof of the target's negation. A failed search over a propositional abstraction of an arithmetic inequality tells us very little about that inequality.

A mathematical counterexample also needs care. A numerically suggested value can be useful feedback, but refutation requires exact checking of the assumptions and the negated conclusion. Floating-point evidence alone is not a proof. Likewise, a solver's raw success or unsatisfiability flag is not automatically an accepted proof object.

9.5 Discover now, replay later

Exploration and publication have different needs. In discovery mode, the system may try alternate lemmas, suggest a stronger induction hypothesis, or propose a new checkpoint. In publication mode, those choices should already be fixed. Open-ended search should not rerun whenever the document is built.

A proof lock record should identify the target expression, selected declaration and instance identities, relevant definitions, the obligation structure, the chosen methods, and the final artifacts. This is not merely a cache of successful prompts. It is a reproducibility record that makes semantic changes visible and allows proof checking without repeating the original search.

An explicit tactic script is better than an unbounded search command, but it may still depend on a changing simplifier or elaborator. A stronger artifact retains the reconstructed declaration and enough environment information for rechecking. Reproducibility is always relative to a pinned toolchain and dependencies; bit-for-bit identical terms across arbitrary Lean versions are not a realistic default requirement.

9.6 Locality and explanation policies

The solver should not have free access to a theorem being proved, to future checkpoints, or to assumptions local to a different case. A policy restricting local facts should be enforced by constructing the obligation in a dependency-closed restricted telescope, not merely by asking a model to ignore other facts.

There are two useful explanation levels. In a *checkpoint profile*, every stated assertion is proved, but the checker does not require the eventual proof term to mirror the author's intended method exactly. In a *method-constrained profile*, a command such as `integrate` must use its declared transformation interface. Both profiles can be sound. The latter offers a stronger account of what the displayed argument actually does, although it may reject a valid alternative proof.

Even the stronger profile should avoid grand claims about human reasoning. Syntactically forcing a proof to mention a lemma does not prove that the lemma was indispensable. The best attainable contract is transparent structure and accurately reported dependencies, not verification of a mathematician's mental process.

10 A small formal semantics

The semantics should be simple enough that its safety properties can be stated independently of a particular search engine. This section gives a propositional core with typed parameters; a full implementation would extend it using Lean's own rules for dependent terms and eliminators.

10.1 Contexts and elaboration

Let $\mathcal{E}$ be a well-formed Lean environment and let Γ be an ordered telescope of local declarations. Write

$$\mathcal{E}; \Gamma \vdash p : P$$

for the kernel typing judgment. A proof block s elaborates according to

$$\mathcal{E}; \Gamma \vdash s \Downarrow p : P \ \triangleright \ \mathcal{O}_1, \ldots, \mathcal{O}_m.$$

Here p is a partial proof expression whose holes are exactly the listed obligations. Each hole has its own local telescope, target, and policy. A successful document has no unresolved holes after reconstruction and is independently rechecked.

The displayed goal P is elaborated and fixed before proof search. The elaborator may discover an expression definitionally equal to P, but it cannot replace P by a merely similar-looking statement. A proposed reformulation Q requires a checked implication $Q \to P$ or equivalence, depending on the operation.

10.2 Core proof rules

For `have h : Q`, the elaborator first constructs $q : Q$ in the current context, then checks the continuation in $\Gamma, h : Q$. The result is a local definition or equivalent term application:

$$\frac{\Gamma \vdash q : Q \qquad \Gamma, h : Q \vdash r : P}{\Gamma \vdash (\lambda h : Q.\, r)\, q : P}.$$

The omitted global environment is the same in each premise. A false checkpoint cannot be bypassed: a term q is required even if r does not mention h. For explanation diagnostics, an unused checkpoint means that its local fact is absent from the continuation's recorded

dependency graph; the final term may still contain a checked but discarded argument. This is not a claim of logical independence.

For a universal goal, `fix x : A` introduces a fresh variable:

$$\frac{\Gamma, x : A \vdash p : P(x)}{\Gamma \vdash \lambda x : A.\, p : \forall x : A,\ P(x)}.$$

For an implication, `assume h : Q` is the analogous introduction rule. It is legal only inside the block whose target is that implication, or inside a separately delimited lemma proving one.

For `suffice Q`, reconstruction supplies both $q : Q$ and $k : Q \to P$, and returns kq. For an existential conclusion, an explicit witness $a : A$ and a proof $p : P(a)$ form $\langle a, p\rangle : \exists x : A, P(x)$. For existential elimination, the witness introduced in a subproof may not escape its scope except through an allowed elimination rule. These are ordinary logical requirements, not optional style restrictions.

A calculation chain composes relation proofs using a selected transitivity theorem. An induction block applies a recursor with a fixed motive. A symmetry block applies the reduction theorem specified in Section 7. Thus the specialized commands add elaboration rules, not new inference axioms.

10.3 How hypotheses generated during refinement are handled

During a subproof, it is often legitimate to reason under an assumption that will later be discharged. It is not legitimate to forget to discharge it. Each local assumption should therefore have an owner: an implication introduction, a case branch, an existential elimination, or another explicit binder-producing construct. Closing the block must produce the corresponding abstraction or elimination term.

An obligation graph may temporarily contain proof holes referenced by downstream nodes. The references must form a well-founded dependency graph. A cycle such as "P because Q; Q because P" produces no certificate merely because both nodes now have a justification label. Induction and recursion are not exceptions implemented by allowing arbitrary cycles: they use the relevant recursor or a checked well-foundedness argument.

For dependent holes, later targets can refer to earlier solutions. Those dependencies must be represented by an ordered dependent telescope and substitutions, with type checking after substitution. A textual substitution scheme that forgets binder identities is insufficient.

Proposition 10.1 (Conditional elaboration soundness)**.** *Assume that the fixed environment $\mathcal{E}$ is accepted by the chosen checker. Suppose a proof block with frozen target P elaborates to a well-typed partial term whose only missing constituents are a finite acyclic family of typed obligations. If every obligation is filled by a term accepted in its exact recorded context, all required substitutions preserve typing, and the resulting closed declaration passes independent checking against P, then the exported declaration proves P relative to $\mathcal{E}$ and its recorded axioms.*

Proof. Order the obligations topologically. For the first obligation, substitute its checked solution for its hole. The typing substitution lemma preserves the surrounding partial term's type. Repeating this argument along the order fills each hole after its prerequisites have been filled; dependent target types are rechecked under the corresponding substitutions. The final expression has no missing constituents and has type P. Equivalently, an induction over the structured proof establishes typing using the core rules above and the checked combiners of domain commands. The final independent check validates the assembled declaration, so an implementation bug in the elaborator or a bad candidate from a provider does not itself become a new logical rule. The conclusion is relative to the environment and the checker, not an assertion of their absolute consistency. □

The proposition is intentionally conditional. It does not prove that a compiler implementing these interfaces exists, that it will terminate quickly, or that it will reconstruct every reasonable mathematical omission. It identifies the obligations a compiler must satisfy to inherit the backend's logical assurance.

10.4 What this theorem does not prove

Kernel typing proves the formal proposition that was checked. It does not prove that the selected definition of continuity, the instance attached to an order symbol, or a translated theorem statement matches the author's intention. Lean's validation guidance explicitly distinguishes theorem validity from the meaning of the statement and recommends stronger checks for untrusted generated code [13].

Mosaic therefore needs a *semantic review boundary* before search. The interface should show a normalized statement with explicit domains, quantifier order, important coercions, and selected definitions. For sensitive applications, the target should be supplied from a separate trusted module. Proof generation must not be able to alter that module.

Moreover, all checked checkpoints could be true while the explanation remains unhelpful. An assertion might be irrelevant or needlessly complicated. Logical soundness and explanatory quality are distinct acceptance criteria: the first is checked mechanically; the second needs structural diagnostics and, ultimately, human evaluation.

11 A Lean-first implementation architecture

11.1 The smallest credible implementation

The first version should be a Lean package adding a restricted proof syntax, a typed obligation representation, and an editor view. It should elaborate expressions through Lean rather than maintain a parallel mathematical type system. Core statements, lexical scopes, named checkpoints, fixed induction motives, and restricted local contexts are enough to test the main idea.

Three initial domain adapters would be especially informative: ordered-field algebra with explicit cancellation conditions; interval comparison and elementary power integrals; and finite counting with natural-to-rational transport. These adapters correspond to the case studies, but their interfaces should be reusable. If each example needs a custom all-powerful tactic, the project has demonstrated bespoke automation rather than a general language architecture.

The adapter output should be a checked theorem application together with child obligations. An informal record saying "integrability established" is not enough. For an integral comparison, the adapter should expose the actual integral operator and measure, interval convention, and theorem selected, so that pointwise, almost-everywhere, improper, and interval-integral reasoning cannot be confused.

11.2 A provider boundary for Leant

A Leant adapter should receive a fully specified goal and an exact local context, not only the sentence shown in the editor. It can request structural candidates, return proposed Lean terms, or suggest tactics for residual goals. Every candidate must be re-elaborated or reconstructed under the authoritative context and checked before it enters the proof graph.

The cross-process boundary deserves special attention. Pretty-printed strings can omit implicit arguments, universe parameters, binder identities, or instance choices. They are useful

for display but should not be the sole semantic identity of a request. A robust protocol needs stable identifiers for declarations and local binders, an explicit dependency environment, and a versioned representation of the relevant terms. An alternative prototype can create a fresh Lean declaration abstracted over the full local telescope and let the backend resolve candidates there. In either case, the returned term is checked against the original target, not against an independently reinterpreted paraphrase.

A logical adapter must also treat negative evidence conservatively. Search exhaustion, unsupported translation, and a proof of negation are separate result constructors. A callback receipt such as the current Leant verification wrapper can help with internal workflow, but the publication boundary must retain and recheck the underlying artifact.

11.3 Suggested internal data model

```
Obligation {
  id                 : StableNodeId
  source_span        : SourceSpan
  environment_id     : EnvironmentFingerprint
  local_telescope    : TypedContext
  target             : ElaboratedExpression
  permitted_inputs   : DependencyPolicy
  method_contract    : OptionalContract
  budget             : ResourceBudget
  prerequisites      : Array StableNodeId
}

AcceptedArtifact {
  target             : ElaboratedExpression
  declaration        : SerializedCheckedDeclaration
  environment_id     : EnvironmentFingerprint
  axiom_dependencies : Array DeclarationId
  checkpoint_map     : Array SourceToCertificateLink
}
```

These are conceptual interfaces, not executable type declarations. In particular, calling a field `SerializedCheckedDeclaration` does not guarantee that bytes really encode a checked term. Acceptance is an operation performed by a checker in an appropriate environment, not a property inferred from a constructor name supplied by another process.

The actual implementation will need more information for universe constraints, local instances, definition transparency, and dependent metavariables. Any cache key omitting a semantically relevant part of that state is a potential source of stale or wrongly scoped results.

11.4 Isolation and trusted computing base

Candidate generation should run without permission to modify the trusted target or the verification environment. Lean metaprograms and external search tools can execute code; they are not automatically harmless because their intended output is a proof. For untrusted generated material, sandboxing and separate artifact checking are therefore part of the design, not just deployment convenience.

A strict publication profile should reject unresolved metavariables, `sorryAx`, undeclared extra assumptions, and axioms outside its allowed policy. That policy need not demand an axiom-free development: standard classical or extensional principles may be intentionally allowed. A

constructive profile can choose a different permitted set, although it must also account for the dependencies of imported mathematics.

The audit should inspect transitive dependencies, not only the theorem's visible source. The current Lean documentation also describes a version-sensitive change: from Lean 4.29.0 onward, native-computation assertions use dedicated axioms rather than relying only on the older `Lean.trustCompiler` mechanism [13]. Consequently, a validator must not blacklist just one historical name and declare all other proofs safe. The appropriate policy is an explicit, version-aware allowlist and a recorded account of any additional trusted computation.

Independent export and checking are stronger than trusting an editor's success indicator. The published trust statement should explain which kernel, environment, axiom policy, serialization path, and, where relevant, additional checker were used. Hashes authenticate identities and help detect changes; they do not prove mathematical correctness.

11.5 Caching, replay, and changes

A safe cache is keyed by the elaborated target, its typed local context, all relevant declarations and instances, the toolchain, and the reconstruction policy. Structural normalization can improve cache hits, but proof equivalence is not a simple general-purpose deduplication test. A cache hit still needs artifact validation in the current authoritative environment.

Changes should be classified. A cosmetic source edit can reuse all mathematical artifacts. A new label may require only source-map updates. A changed intermediate statement invalidates its downstream obligations. A changed definition or instance may alter the meaning of the target and must trigger semantic review, not merely fresh proof search.

Cross-version migration should be explicit. A migration tool can propose replacement lemmas and regenerate proofs, but it should display statement and dependency differences and require the same checking as an original construction. A successful recompile of altered text is not, by itself, evidence that the intended theorem has remained unchanged.

11.6 Publishing and progressive disclosure

The printed article should show the spine by default. An interactive document can reveal the ledger and certificates on demand. A literate-document framework such as Verso provides an existing Lean-oriented direction for document tooling, though integration with the proposed proof graph would be new work [26].

The proof display should have distinct states: parsed, semantically elaborated, partially reconstructed, kernel checked, and audited for publication. A single green icon for all of these would conceal exactly the distinctions the architecture is meant to protect. A prose annotation can remain useful even when it has no formal semantics, but it must not inherit the status of a checked assertion merely by proximity.

12 Failure should be explained mathematically

12.1 A useful diagnostic has a local contract

When reconstruction fails, a generic message such as "no tactic found" is not enough. The system should show the intended operation, the instantiated theorem or schema, the facts already established, and the residual obligation. It should distinguish a missing premise from unsupported syntax and exhausted search.

For the flatness proof, a useful message would be:

```
The induction hypothesis is available only at x.
This step requests it at 2*t.
Suggested change: generalize x in the induction motive.
No change has been applied to the theorem or proof.
```

For natural division, the message should identify the proposed cast theorem and the missing divisibility proof. It may suggest a sum-of-squares identity as a source of evidence. It must not silently switch to rational division inside the original natural-valued statement.

A "smallest missing obligation" diagnostic can itself be difficult to compute, especially when many lemmas are possible. The implementation should report the residuals from its chosen reconstruction attempt and avoid claiming that they are globally minimal or logically necessary.

12.2 Adversarial examples for the language contract

The test suite should include short plausible-looking arguments that must fail. For example, cancelling x from $xy = xz$ without $x \neq 0$ must leave a nonzero obligation. Integrating an inequality over reversed endpoints must not preserve its direction without explanation. Replacing an almost-everywhere identity by a value at a specified exceptional point must fail. A symmetry reduction that changes a hypothesis must fail. Two checkpoints that justify one another cyclically must remain incomplete.

Likewise, a proof of a weakened statement must not satisfy the original target. A proposition whose notation has been rebound in an untrusted generated module must not evade a separately fixed challenge statement. A successful local proof depending on a forbidden axiom must fail the publication audit. A cached result from a different instance context must be rechecked rather than reused by textual similarity.

One important test is more subtle: insert a false checkpoint into a document whose final theorem is trivial. The theorem-only solver may still succeed, but the document checker must reject the unproved checkpoint. Another is a true but unused checkpoint: the document may be accepted logically, while the explanation view accurately marks that checkpoint as unused. These two cases should not be conflated.

12.3 "Sufficiently small" as a synthesis contract

Suppose the author has proved

$$\forall \varepsilon > 0\ \exists \delta_f > 0\ \forall x,\ |x - c| < \delta_f \Rightarrow |f(x) - f(c)| < \varepsilon/2$$

and the analogous statement for g. For a fixed $\varepsilon > 0$, "choose δ sufficiently small for both bounds" can be elaborated into

$$\exists \delta > 0, \qquad \delta \leq \delta_f \ \wedge\ \delta \leq \delta_g.$$

The witness $\min(\delta_f, \delta_g)$ solves this contract, and its positivity and two comparison properties are straightforward obligations. If $|x - c| < \delta$, both error estimates apply, so the triangle inequality yields

$$|(f + g)(x) - (f + g)(c)| \leq |f(x) - f(c)| + |g(x) - g(c)| < \varepsilon.$$

Here synthesis can choose an uninteresting witness without hiding the mathematical idea: split the error budget and satisfy both local bounds. But it must not choose δ before ε is fixed, infer a positive minimum without positivity of its inputs, or substitute a non-strict bound where strictness is needed. This example suggests a future library of *witness contracts*, not a primitive that treats the phrase "sufficiently small" as self-justifying.

12.4 Constructive and classical modes

The same human argument may have several formal implementations. An explicit minimum needs no choice principle. Selecting an object from a nonempty arbitrary type may use classical choice. Proving an implication by contradiction may require a classical principle, depending on the proposition and the available assumptions.

A logic profile should affect both provider selection and the final dependency audit. It cannot be enforced solely by removing a visible `classical` keyword: an imported theorem may already depend on classical principles. Conversely, not every use of case analysis is classical; a decidable proposition can support constructive branching. The diagnostic should identify the actual additional principle required by the attempted proof.

13 Implementation roadmap and an honest evaluation

13.1 An incremental roadmap

Stage	Deliverable	Evidence required before moving on
Core	Typed checkpoints, scoped assumptions, explicit target locking, ordinary Lean escape blocks.	Tests for binder hygiene, rejected false checkpoints, closed obligations, and exact exported targets.
Adapters	Field/order algebra, certified numeric views, interval comparison, and finite counting interfaces.	Positive and negative tests for every side condition; proof artifacts for adapter theorems.
Synthesis	Leant-style structural provider, selected Lean tactic providers, bounded discovery, and stable replay.	Candidate-to-target checks, conservative failure statuses, session-restart tests, and no discovery required in replay mode.
Research UI	Progressive disclosure, statement diffs, dependency explanations, and literate export.	User studies and maintenance experiments on held-out proofs, including an idiomatic Lean baseline.

Table 3: A dependency-ordered roadmap, not a promised schedule. Each stage has a falsifiable completion criterion.

The first milestone should not be a full natural-language parser or an autonomous research prover. It should be a small reliable core that makes the obligations visible. A surprisingly modest front end, integrated with strong existing tactics and a few carefully designed adapters, could already test the central claim.

Leant integration can initially be optional. This avoids making the proof language depend on one search engine's fragment restrictions or process protocol. Conversely, Leant can benefit from the language's typed checkpoints as smaller, better-specified synthesis requests. The two projects can cooperate without sharing a new trusted kernel.

13.2 Baselines that do not rig the outcome

Evaluation should compare at least five configurations: the original source; an idiomatically refactored Lean version using the same mathematical helper lemmas; declarative syntax without additional automation; broad automated proof search; and the full contract-directed design. The same environment and foundational policy should be used wherever possible.

The purpose of the refactored baseline is crucial. A language should not take credit for shortening a proof by moving most of it into an uncounted custom lemma. Conversely, a reusable adapter has legitimate amortized value, so it should not be charged in full against every theorem. Report both cold-start cost and cost over a growing suite.

The proof sample should include elementary logic, arithmetic with coercions, finite combinatorics, basic analysis, and at least one genuinely dependent construction. The three source files in this article are starting points, not a representative benchmark. Hold out related theorems to test whether adapters generalize beyond the examples for which they were designed.

13.3 What to measure

Surface length is useful but insufficient. Measure author editing time, accepted mathematical checkpoints, newly written infrastructure, reconstruction failures, search cost, replay cost, certificate size, and maintenance effort after controlled changes. Also measure how often the system proposes an unintended statement interpretation and whether reviewers notice the difference.

For a suite of m proofs, a transparent accounting of source burden is

$$C_{\text{suite}} = L_{\text{spines}} + L_{\text{new adapters}} + L_{\text{bespoke support}} + L_{\text{handwritten escape blocks}}.$$

This is not a universal quality score; line counts depend on formatting. Its purpose is to prevent hidden work from disappearing from the comparison. Token counts, editing actions, and a separate mathematical-complexity classification can complement it.

A human study should ask participants to explain the proof, identify which hypothesis is used at a selected step, detect an invalid transformation, and modify the theorem or argument. Randomize presentation order and account for participants' familiarity with Lean and the mathematical domain. A shorter proof that reduces comprehension or makes mistakes harder to locate is not an unqualified success.

For robustness, rename local variables, reorder irrelevant facts, change notation without changing meaning, and upgrade a dependency under an explicit migration protocol. Distinguish harmless presentation changes from semantic ones. Successful reuse should preserve the frozen interpretation, not merely find another theorem with similar printed text.

13.4 Hypotheses, not fabricated results

The design suggests three testable hypotheses. Contract-directed reconstruction may reduce representation-heavy editing after adapters are available. Checked checkpoints may improve explanations compared with a single opaque proof-search command. Frozen statements and isolated replay may reduce accidental semantic drift in AI-assisted workflows.

None of these claims is reported here as an experimental result. This article contains no measured compression ratio, compile-time improvement, user-study outcome, or newly checked proof implementation. A useful first experiment is whether the analytic and arithmetic case studies can be expressed with the proposed contracts without accumulating more adapter-specific complexity than the equivalent refactored Lean library.

14 Limits, open design questions, and recommendation

14.1 What a new language cannot remove

A missing mathematical lemma remains missing. Better phrasing cannot make a difficult compactness argument, invariant, asymptotic estimate, or construction appear automatically. A sufficiently ambitious search engine may discover such material, but the language must then present it as newly proposed mathematics, not routine elaboration of an obvious step.

Nor can the system universally infer the author's intended definitions from notation. Mathematical conventions vary; a finite sum may include or exclude an endpoint, a derivative may be within a set, and a quotient may belong to several different algebraic structures. These choices belong in the semantic review layer even when the displayed syntax is familiar.

A third limit is the tradeoff between automation and stability. Larger search spaces can close more omissions but make failures less predictable and replay more expensive. Stronger method constraints produce more faithful explanations but can reject a perfectly valid alternative proof. User-selectable profiles can expose this tradeoff, but they do not eliminate it.

14.2 Promising research questions

One research direction is *adaptive granularity*: learn where an author needs another checkpoint, while verifying any proposed refinement before it becomes part of the document. Another is *proof-relevant retrieval*: select lemmas by the operation they justify and the obligations they leave, not only by textual similarity. A third is *certified view composition*: compose numerical, set-theoretic, and functional representations while keeping their side conditions understandable.

A fourth direction concerns explanation itself. Can an interface identify a small, useful collection of intermediate assertions that connects a long certificate to an understandable argument? Such extraction would be an optimization over presentation, not a new theorem about correctness. It should be evaluated against human explanations and explicitly distinguish omitted but checked details from unsupported commentary.

Finally, machine learning could improve obligation scheduling and suggestion ranking. Those learned components should remain replaceable and untrusted. Their outputs need the same typed contracts and checking as deterministic providers. The more powerful discovery becomes, the more valuable a stable boundary between conjecture, proposed formalization, checked proof, and reviewed mathematical meaning will be.

14.3 The practical recommendation

Do not begin by replacing Lean with unrestricted mathematical English. Begin with a Lean-native declarative core that exposes the mathematical spine and treats omitted details as scoped reconstruction obligations. Add a few proved domain interfaces, a Leant adapter for structural synthesis, and a publication path that freezes statements and checks artifacts independently.

The desired end state is not that all proofs have one line. It is that the author spends lines on the choices that make the proof work, while the system accounts for every omitted detail. A human should be able to read why the theorem is true; a machine should be able to check exactly what is claimed; and both should be able to inspect the bridge between those two views.

A Pinned source manifest and limits of inspection

The following revisions identify the repository evidence used in this design study. They are not claims about the state of every file on a moving branch after the review date.

ProveIt
`24ce8bd743eaab64a91ce90725ea00f498d319d2`

Leant
`c36adf114035fb2fba6c276b63944cc613b31668`

Mathematical source files

`Analysis/FabiusFunction/Lean/FabiusFunction/SharpFlatness.lean`

The central declaration is `fabiusReal_le_two_pow_div_factorial_mul_pow`. The integral identity and transfer corollary supply context for the design example. The article's analytic interface and proof spine are proposed reformulations, not generated or compiled replacements.

`Analysis/FabiusFunction/Lean/FabiusFunction/Regularity.lean`

The inspected material includes Lipschitz estimates, sharpness at the midpoint, and the absolute-value linear bound. The discussion deliberately notes both long proofs and a short corollary; it does not assert uniform verbosity throughout the repository.

`NumberTheory/IntegerSums/Lean/IntegerSums/FloorSqrtSum.lean`

The inspected file includes natural and rational closed forms, successor-step lemmas, divisibility and subtraction bounds, the cast bridge, and the exported natural equality. The double-counting derivation in this article is an alternative proposed proof.

Synthesis and interaction sources

`README.md`

`docs/synth-internals.md`

`src/Leant/Synth/Fragment.hs`

`src/Leant/Synth/Verification.hs`

`src/Main.hs`

The review used selected sections concerning fragment translation, verification wrappers, candidate/evidence handling, tactic suggestion, and script recording. It is not a full Haskell code audit, a security audit, or an empirical validation of all behavior documented by Leant.

Verification status of this artifact

The LaTeX article was built and its PDF inspected. The mathematical derivations are supplied as ordinary proofs. No Lean kernel was run on the proposed programs, and the archive makes no assertion that Mosaic syntax currently parses or that the proposed adapters exist. Bibliographic sources support descriptions of existing systems; architecture, semantics, examples, and roadmap are the proposal of this article.

B Compact grammar and obligation contracts

The following is a deliberately incomplete abstract grammar for the proposed core. It fixes the distinction between statements, proofs, and justifications while leaving expressions to the underlying typed language. The examples in the body also use domain extensions.

```
proof       ::= "proof" step* "end"
step        ::= "fix" binder
              | "assume" label ":" proposition
              | "let" label ":" type ":=" expression
              | "have" label ":" proposition justification
              | "show" proposition justification
              | "suffice" label ":" proposition justification
              | "obtain" binders "from" reference
              | "cases" expression branch+
              | "induction" expression generalization? branch+
              | calculation
              | domain_command
justification ::= "by" method scope?
                | "by Lean" lean_block
                | proof
scope       ::= "using" references
              | "from only" references
              | scope "library only" declaration_set
```

A complete grammar must also specify precedence, branch delimiters, the exact meaning of a goal label, and how domain extensions are registered. It should avoid synonym proliferation in its canonical saved form. The editor can accept convenient surface variants, but canonicalization should make diffs and replay understandable.

Operation	Minimum contract
`algebra`	Fix the algebraic structure and transformations; produce equality or order proofs; discharge every cancellation and denominator condition.
`transport`	Identify the source and target views; instantiate preservation theorems; discharge side conditions; provide a map back to the original target.
`integrate`	Fix the integral, domain, orientation, and measure; prove the required comparison and integrability conditions; return the precise integral inequality.
`symmetry`	Prove domain preservation, coverage of the selected cases, and transport of the whole target including its hypotheses.
`count`	Fix a finite representation; supply a bijection or fiber decomposition; prove membership equivalences and fiber-cardinality formulas.
`induction`	Fix the recursor and motive; state generalized parameters; construct every required branch under the permitted induction hypotheses.
`choose`	State the witness property; prove existence or construct the witness; respect dependencies and any required choice principle.

Table 4: The proof obligations that make mathematical operations more than suggestive keywords.

C Acceptance checklist for a future prototype

Before a document is labeled accepted, the implementation should establish that its theorem target matches the trusted elaborated statement; every authored checkpoint has a checked proof in the correct context; every domain transformation has its required side conditions; every

witness, instance, and coercion has a recorded interpretation; and the final declaration has no unresolved proof holes or forbidden axiom dependencies.

Before it is labeled reproducible, replay should work from pinned dependencies without remote model calls or unrestricted discovery. Before it is labeled explanation-preserving, the display should distinguish used from unused checkpoints, identify the selected method contracts, and allow the reader to inspect the obligation graph. These additional labels are separate from ordinary kernel acceptance.

Before claiming an improvement over Lean, the project should publish its refactored Lean baselines, adapter sources, negative tests, resource budgets, failures, and human-evaluation protocol. The goal is a credible advance in mathematical authoring, not a demonstration that omitted implementation text takes fewer characters to print.

References

[1] V. Reshetnikov, *ProveIt*, repository snapshot, commit `24ce8bd743ea`, September 5, 2026. https://github.com/VladimirReshetnikov/ProveIt/commit/24ce8bd743eaab64a91ce90725ea00f498d319d2.

[2] V. Reshetnikov, *Leant*, repository snapshot, commit `c36adf114035`, September 5, 2026. https://github.com/VladimirReshetnikov/Leant/commit/c36adf114035fb2fba6c276b63944cc613b31668.

[3] V. Reshetnikov, ProveIt `lean-toolchain`, at the revision in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/lean-toolchain.

[4] V. Reshetnikov, ProveIt, `SharpFlatness.lean`, at the revision in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/SharpFlatness.lean.

[5] V. Reshetnikov, ProveIt, `Regularity.lean`, at the revision in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/Regularity.lean.

[6] V. Reshetnikov, ProveIt, `FloorSqrtSum.lean`, at the revision in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/NumberTheory/IntegerSums/Lean/IntegerSums/FloorSqrtSum.lean.

[7] V. Reshetnikov, Leant, `README.md`, at the revision in [2]. https://github.com/VladimirReshetnikov/Leant/blob/c36adf114035fb2fba6c276b63944cc613b31668/README.md.

[8] V. Reshetnikov, Leant, `src/Leant/Synth/Fragment.hs`, at the revision in [2]. https://github.com/VladimirReshetnikov/Leant/blob/c36adf114035fb2fba6c276b63944cc613b31668/src/Leant/Synth/Fragment.hs.

[9] V. Reshetnikov, Leant, `src/Leant/Synth/Verification.hs`, at the revision in [2]. https://github.com/VladimirReshetnikov/Leant/blob/c36adf114035fb2fba6c276b63944cc613b31668/src/Leant/Synth/Verification.hs.

[10] V. Reshetnikov, Leant, `src/Main.hs`, especially `suggestTactic` and proof-script recording, at the revision in [2]. https://github.com/VladimirReshetnikov/Leant/blob/c36adf114035fb2fba6c276b63944cc613b31668/src/Main.hs.

[11] V. Reshetnikov, Leant, `docs/synth-internals.md`, at the revision in [2]. https://github.com/VladimirReshetnikov/Leant/blob/c36adf114035fb2fba6c276b63944cc613b31668/docs/synth-internals.md.

[12] Lean developers, *The Lean Language Reference: Tactic Proofs*. Online documentation, accessed September 5, 2026. https://lean-lang.org/doc/reference/latest/Tactic-Proofs/.

[13] Lean developers, *The Lean Language Reference: Validating a Lean Proof*. Online documentation, accessed September 5, 2026. Version-sensitive details must be checked against the selected toolchain. https://lean-lang.org/doc/reference/latest/ValidatingProofs/.

[14] Mizar project, *Mizar Language*. Official language resources, accessed September 5, 2026. https://mizar.uwb.edu.pl/language/.

[15] Isabelle project, *Isar: Intelligible Semi-Automated Reasoning*. Official project description, accessed September 5, 2026. https://isabelle.in.tum.de/Isar/.

[16] F. Wiedijk, "A Synthesis of the Procedural and Declarative Styles of Interactive Theorem Proving," *Logical Methods in Computer Science* 8(1:30), 2012, pp. 1–26. https://lmcs.episciences.org/1046. https://doi.org/10.2168/LMCS-8(1:30)2012.

[17] Naproche project, official project description and repository, accessed September 5, 2026. https://naproche.github.io/. https://github.com/naproche/naproche.

[18] A. De Lon et al., "The Isabelle/Naproche Natural Language Proof Assistant," in *Automated Deduction—CADE 28*, 2021. https://doi.org/10.1007/978-3-030-79876-5_36.

[19] P. Massot, "Teaching Mathematics Using Lean and Controlled Natural Language," in *ITP 2024*, LIPIcs 309, 27:1–27:19, 2024. https://doi.org/10.4230/LIPIcs.ITP.2024.27.

[20] P. Massot and contributors, *Verbose Lean 4: Natural Language Tactics to Teach Mathematics Using Lean 4*. Repository, accessed September 5, 2026. https://github.com/PatrickMassot/verbose-lean4.

[21] Waterproof developers, *Waterproof*, official proof-assistant component repository, accessed September 5, 2026. https://github.com/impermeable/coq-waterproof.

[22] X. Zhai, X. Chen, Y. Wang, R. Zhou, L. Zhang, and S. S. Du, "Visored: A Controlled-Natural-Language Prover for LLM-Generated Mathematics," arXiv:2606.17581v1, June 16, 2026. Preprint; cited for its stated architecture, not as an independently validated benchmark. https://arxiv.org/html/2606.17581v1.

[23] Lean community, *Aesop*, official repository and documentation, accessed September 5, 2026. https://github.com/leanprover-community/aesop.

[24] Lean developers, *The Lean Language Reference: The grind Tactic*. Online documentation, accessed September 5, 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/.

[25] S. Welleck and R. Saha, "LLMSTEP: LLM Proofstep Suggestions in Lean," arXiv:2310.18457, 2023. https://arxiv.org/abs/2310.18457.

[26] Lean developers, *Verso*, official repository, accessed September 5, 2026. https://github.com/leanprover/verso.